Scripting reference

Table of contents

Namespaces and assemblies

Core runtime types are in namespace `EasyIdleGame` and assembly `EasyIdleGame`.

uGUI displayers, audio helpers, demo managers, and UI Toolkit presenters are in `EasyIdleGame.UI` and assembly `EasyIdleGame.UI`.

Editor-only inspectors, menu commands, and tests compile into `EasyIdleGame.Editor` or test assemblies and must not be referenced by player runtime code.

If your own asmdef references UI types, reference both `EasyIdleGame` and `EasyIdleGame.UI`.

This maintained reference and the current source are the authoritative API documentation. Use the interface, operation, event, enum, and core-type indexes below to locate the supported integration surface.

Access managers and holders

Managers are scene components exposed through a static `Instance` property:

```
CurrencyManager currencies = CurrencyManager.Instance;
BusinessesManager businesses = BusinessesManager.Instance;
```

`CurrencyManager` and `BusinessesManager` are marked necessary and throw when accessed without a scene instance. An absent optional manager returns `null`; guard optional integrations before dereferencing it.

Resolve runtime state through a manager:

```
BusinessHolder holder = BusinessesManager.Instance.GetOrAddHolder(business);

if (UpgradesManager.Instance.TryGetHolder(upgrade, out UpgradeHolder upgradeHolder))
{
```

```
    BigNumber activeLevels = upgradeHolder.Amount;
}
```

`GetOrAddHolder` creates default state. Prefer `TryGetHolder` when a read should not add a previously untracked holder.

Managers clear serialized holder lists in `Awake`; saved holders are restored by `SaveFile.Load()`.

Extend holders and managers

Use the supplied base classes only when a project-specific system genuinely needs package-style definition, holder, and manager layers:

- Derive a serializable runtime holder from `HolderBase<TDefinition>`. Its constructor must pass the definition asset to `base(item)`, and its mutable fields must be included in the project's save model.
- Derive a singleton scene component from `ManagerBase<TManager>` when the system does not own holders. Optional managers return `null` from `Instance` when absent; set the protected static `_necessary` flag only when missing the component must be a hard configuration error.
- Derive a holder-owning component from `ManagerHolderBase<TManager, THolder, TDefinition>` when the system manages a collection of definition-backed holders. Implement `GetOrAddHolder`, raise `OnHolderAdded` when a new holder is created, and define reset notifications when other systems depend on them.
- Keep definition assets below a Resources folder when they must be discovered or restored by name. Give same-type assets unique, stable names.

The base manager clears its runtime holder list in `Awake`; load saved state only after scene managers are initialized. Prefer composing a project system around the public manager APIs when it does not need to participate in package persistence, prestige resets, generic storefronts, or capability interfaces.

Use capability interfaces

Definition and holder types expose capabilities through interfaces:

Definition capability	Holder capability	Purpose
`IBuyable`	`IBuyableHolder`	Affordability, payment, geometric cost, limits
`IUnlockable`	`IUnlockableHolder`	Lower/upper locks and unlock-once state
`IProducable`	`IProducableHolder`	Production cost, rounds, output calculation, collection
`IUpgradable`	`IUpgradableHolder`	Upgrade levels and level-up cost
`ILevelupable`	`ILevelupableHolder`	XP/copy levels and rewards
`IMultiplierProvider`	`IMultiplierProviderHolder`	Upgrade-block scaling by owned amount

Concrete types expose adapters such as `business.iBuyable`, `holder.iBuyable`, and `holder.iProducable`. Use the capability interface when writing shared code across businesses, managers, boosts, upgrades, locations, or shop items.

Each capability also has a manager-side contract — `IBuyableHolderManager`, `IUnlockableHolderManager`, `ILevelupableHolderManager`, `IUpgradableHolderManager`, `IProducableHolderManager`, and `IMultiplierProviderHolderManager` — exposed through manager adapters such as `BusinessesManager.Instance.iBuyable` and `ManagersManager.Instance.iBuyable`. These carry the generic operations (`TryBuyItems`, `GetItemAmount`, `IsItemUnlocked`, `GetCurrentItemLevel`, purchase previews) so one storefront or list screen can serve businesses, managers, boosts, upgrades, locations, and shop items.

Other public interfaces to know:

Interface	Purpose

`IMetadataProvider`	Display name, icon, icon string, model, and description access on definition assets
`ILocatable`	Associates a business with a `Location` for location-scoped availability
`IMergableBoost`	`TryMergeBoost(self, other)` decides whether a new activation merges into an existing active holder
`IMultipliableBoost`	Marks a boost that contributes upgrade-block multipliers while active
`IProductionOutputModifier` / `IOrderedProductionOutputModifier`	Output-stage production modification and its explicit ordering (`Priority`)
`ILogger`	Manager diagnostics routed through `LogCategory`
`IOpenable`, `IMetadataDisplayer` (in `EasyIdleGame.UI`)	Openable panel and metadata-binding contracts used by the supplied displayers

Common operations

Currency

```
CurrencyManager.Instance.AddCurrencyAmount(
    coin,
    100,
    SourceType.Milestone);

BigNumber current = CurrencyManager.Instance.GetCurrencyAmount(coin);
BigNumber lifetime = CurrencyManager.Instance.GetTotalCurrencyAmount(coin);
```

Business purchase and free grant

```
bool bought = BusinessesManager.Instance.iBuyable
    .TryBuyItems(business, 10, SourceType.Purchase);

BusinessesManager.Instance.ForceBuyItemsForFree(
    business,
    5,
    SourceType.Milestone);
```

The purchase path updates bought amount. A free grant updates total/current state without being a purchase.

Purchase preview

```
PurchasePreview preview = BusinessesManager.Instance.iBuyable
    .GetPurchasePreview(business, 10);

buyButton.interactable = preview.canAfford;
maxLabel.text = preview.maxAffordableAmount.ToString();
```

Use the preview for cost and disabled-button UI; it does not create a holder or mutate state.

BigNumber cost math

```
BigNumber balance = CurrencyManager.Instance.GetCurrencyAmount(coin);
BigNumber price = new BigNumber(1.5, 6); // 1.5 × 10^6

if (balance >= price)
{
    // Compare and calculate as BigNumber; convert only at an API boundary.
}

// Price of the next `buyCount` items on a geometric curve starting at `bought`:
BigNumber first = Mathb.GeometricSequence_NthTerm(baseCost, costMultiplier, bought);
BigNumber total = Mathb.GeometricSequence_Sum(first, costMultiplier, buyCount);
```

Use `Mathb.GeometricSequence_Amount` for "how many can I afford," and clamp before converting a `BigNumber` to a primitive type.

Merge

```
bool merged = BusinessesManager.Instance.TryMergeBusinesses(
    recipe,
    out List<Output> outputs);
```

Travel

```
bool traveled = LocationsManager.Instance.TryTravelTo(location);
```

Shop completion

```
void OnPlatformPurchaseConfirmed(ShopItem item)
{
    ShopManager.Instance.CompleteExternalPurchase(
        item,
        SourceType.Purchase);
}
```

Prestige

```
bool prestiged = PrestigeManager.Instance.TryPrestigeGame(
    out List<Output> rewards,
    out ResetSummary summary);
```

Offline progress

```
ProfitApplicationSummary summary =
    ProfitCalculator.CalculateAndApplyProfit(TimeSpan.FromHours(1));
```

Try-method index

`Try...()` methods are the supported mutating entry points. They validate state, return `false` instead of throwing when a prerequisite fails (the reason is logged at `Warning`), and raise the normal events and save-worthy notifications on success. Methods

with an `out List<Output>` parameter return the actually granted outputs — use them for reward UI and analytics instead of re-rolling tables.

Method	Owner	Returns `false` when
`TryBuyItems(item, amount, sourceType)`	manager `iBuyable` adapters	Cost is unaffordable, the item is locked, or a cap/capacity blocks the amount
`TryGetHolder(item, out holder)`	every holder manager	No holder exists yet; unlike `GetOrAddHolder`, it never creates state
`TryStartProduction()`	`BusinessHolder`	A non-concurrent round is active, production locks fail, no idle copies exist, or inputs are unaffordable
`TryProduceManually(instances, out round, context)`	`BusinessHolder`	Same production checks for an explicit instance count; on success it outputs the started `ProductionRound`
`TryMergeBusinesses(recipe, out outputs)`	`BusinessesManager`	Inputs are unaffordable or the recipe is invalid; `ForceMergeBusinesses` skips validation and returns applied outputs
`TryGetMergeRecipe(a, b, out recipe)` / `TryGetMergeRecipe(inputs, out recipe)`	`BusinessesManager` (static)	No cached recipe matches; refresh with `RefreshMergeRecipeCache()` after runtime catalog changes
`TryUseBoost(boost)`	`BoostsManager`	Inventory is empty; on success it consumes one inventory unit and activates the boost
`TryActivateManager(manager)` / `ManagerHolder.TryActivate()`	`ManagersManager` / `ManagerHolder`	The manager is at level zero or its cooldown is running
`TryUpgrade(sourceType)` / `TryUpgradeToLevel(level, sourceType)`	`IUpgradableHolder` (businesses, managers)	The level-up cost is unaffordable or the target level is invalid; both pay the cost, unlike the `Force...` variants
`TryUnlockLocation(location)`	`LocationsManager`	The location is already owned or its cost/locks block purchase; success pays once and applies starter outputs
`TryTravelTo(location)`	`LocationsManager`	The location is not owned and cannot be unlocked; success changes the active location
`TryGetActiveLocation<T>(out location)`	`LocationsManager`	No active location of the requested subclass exists
`TryClaimAchievement(achievement, out outputs)` / `AchievementHolder.TryClaimRewards(out outputs)`	`AchievementsManager` / `AchievementHolder`	The current tier is not claimable
`TryClaimDailyRewards(day, out outputs)`	`DailyRewardsManager`	The day is already claimed, non-positive, in the future, or has no configured rewards
`TryConsumeItem(item, amount, out outputs, sourceType)` / `ShopItemHolder.TryConsumeAmount(...)`	`ShopManager` / `ShopItemHolder`	The amount is not positive or the holder lacks it; success grants the reward tables again — see the shop guide before combining with completion
`TryConsumeCurrentOffer()`	`OffersManager`	No offer is active; success ends the offer without raising `OnOfferExpired`

`TryPrestigeGame(out outputs)` — plus overloads adding `PrestigeResetOptions` and `out ResetSummary`	`PrestigeManager`	Prestige requirements fail or the maximum count is reached

Force-variant counterparts — `ForceBuyItemsForFree`, `ForceMergeBusinesses`, `ForceUpgradeToLevel`, and `CompleteExternalPurchase` — bypass the corresponding payment or validation step; they are for designed grants and verified external completions, not error handling.

Important events

Subscribe and unsubscribe with the lifetime of the receiving object.

Event	Raised when
`EconomyChangedEvents.OnSaveWorthyStateChanged`	A debounced state transition should be considered for saving
`CurrencyManager.OnCurrencyAmountChanged`	A currency balance changes
`CurrencyManager.OnCurrencyAmountAddedWithSourceEvent`	Currency is added with a `SourceType`
`BusinessesManager.OnBusinessAmountChanged`	Business amount changes
`ManagerHolderBase.OnHolderAdded`	A manager creates a new runtime holder
`UpgradesManager.OnUpgradeAmountChanged`	Active upgrade amount changes
`UpgradesManager.OnUpgradeApplied`	A timed pending upgrade becomes active
`BoostsManager.OnTimeSkip`	A time-skip boost applies profit
`LocationsManager.OnLocationChanged`	Active location changes
`ShopManager.OnAttemptRealMoneyPurchase`	Runtime requests platform IAP
`ShopManager.OnAttemptAdPurchase`	Runtime requests rewarded-ad flow
`ShopManager.OnShopItemRewardsGranted`	A completed shop flow grants outputs
`OffersManager.OnOfferBecameAvailable`	A valid weighted offer activates
`DailyRewardsManager.OnDailyRewardClaimed`	A calendar day is claimed
`PrestigeManager.OnPrestige`	A successful prestige finishes
`ProfitCalculator.OnProfitApplied`	Offline/time-skip `ProfitData` is applied

Prefer source-aware events when analytics, modifiers, or UI distinguish purchases, production, recurring rewards, resets, and random drops.

Coalesce autosaves with EconomyChangedEvents

Subscribe once to `EconomyChangedEvents.OnSaveWorthyStateChanged` to mark project state dirty or queue an autosave. Notifications are debounced per frame: treat the callback as "state became dirty," not as a transaction log, because several same-frame changes produce one notification. `SaveWorthySourcesThisFrame` lists the source strings collected in the current frame, `SuppressedSaveWorthyChangesThisFrame` counts the coalesced notifications, and `ResetSaveWorthyDebounce()` clears static state between isolated tests.

Coalesce disk writes — especially on mobile — and suppress the handler while loading so partial state does not trigger recursive saves. Package operations report their source strings automatically; project-only state changes must call `NotifySaveWorthyStateChanged(source)` or use their own dirty channel.

Production output APIs

`IProducableHolder` exposes real, preview, and optimistic output calculations. The calculation system uses `ProductionOutputContext` to describe:

- Producing holder/business.
- Owned and currently producing amounts.
- Active, offline, preview, and manual state.
- Drop evaluation mode.
- Production multiplier and time.

`ProductionOutputCalculationMode` distinguishes actual/preview/per-second/offline paths. Use the existing holder API to create the context; do not construct a partial context unless you are extending the resolver.

Production modifier assets and contextual upgrade blocks receive the same semantic context, which keeps live UI previews and offline calculations aligned.

Create a custom boost

Derive a ScriptableObject from `Boost` and implement `Activate()`.

Return an `ActiveBoostHolder` for a timed effect or `null` for an immediate effect:

```
using UnityEngine;

[CreateAssetMenu(menuName = "My Game/Boosts/Instant Token")]
public sealed class InstantTokenBoost : Boost
{
    public Currency token;
    public BigNumber amount = 1;

    public override ActiveBoostHolder Activate()
    {
        CurrencyManager.Instance.AddCurrencyAmount(
            token,
            amount,
            SourceType.Event);

        return null;
    }
}
```

Implement `IMergableBoost` when matching active instances should combine. Implement `IMultiplierProvider` when the boost supplies upgrade blocks. The built-in `GenericMultiplierBoost` and `AutoProduceBoost` are the best current examples.

Create a custom production modifier

For output calculation that is not expressible as an upgrade block:

1. Derive from `ProductionOutputModifierAsset` or use the supplied scaling modifier as a template.
2. Read `ProductionOutputContext` instead of global scene state where possible.
3. Return deterministic output for preview/offline modes unless randomness is explicitly part of the feature.
4. Register the asset through `ProductionModifierManager` or `ProductionModifierRegistry`.
5. Test live, per-second preview, manual, and offline paths.

Production modifier registration is separate from generic reward drop-table generation.

Create a custom save file

Derive from `SaveFile` and add serializable fields. The constructor captures state; `Load()` restores it after `base.Load()`.

```
[Serializable]
public sealed class MySaveFile : SaveFile
{
    public int selectedSkin;

    public MySaveFile()
    {
        selectedSkin = MyCosmetics.SelectedSkin;
    }

    public override void Load()
    {
        base.Load();
        MyCosmetics.SelectedSkin = selectedSkin;
    }
}
```

Use a custom component to select the generic serializer:

```
public sealed class MySaveAndLoad : SaveAndLoad
{
    public override void Save()
    {
        SaveAndLoadSystem<MySaveFile>.Save(Path);
    }

    public override void Load(out SaveFile file)
    {
        SaveAndLoadSystem<MySaveFile>.Load(Path, out MySaveFile typedFile);
        file = typedFile;
    }
}
```

If you override `Load`, reapply the package's offline-time policy explicitly; the base `SaveAndLoad.Load` method performs offline calculation after deserialization.

A complete working example — including load-guarded, save-worthy-event-driven autosave — is in `EasyIdleGame > Demos > FeatureDemos > CustomSaveAndAutosave > CustomSaveAndAutosave.unity`.

Create custom UI

Subclass a supplied displayer, or build independent uGUI/UI Toolkit code that reads managers/holders and invokes manager operations. The redraw contract, subclassing example, presenter scope, and event-refresh rules are in UI and displayers (`08-ui-and-displayers.pdf`).

Validate public API inputs

Several public methods assume trusted game code rather than validating every argument. At integration boundaries:

- Reject null definition assets and recipes.
- Require positive amounts for buy, add, remove, and merge flows.

- Use whole-number counts for shop completion; fractional amounts are floored, non-positive requests are rejected, and bulk completions are clamped to the remaining `maxPurchases` amount by the package.
- Daily reward claims are validated by the package: non-positive, future, already-claimed, and rewardless days return `false`.
- `ProfitCalculator.CalculateProfit` returns an empty result for zero or negative elapsed time.

The focused demos use controlled inputs. Platform purchases, remote configuration, save migrations, and player-entered debug tools must add their own validation.

Implement a custom package contract

A custom `IBuyable`, `IUnlockable`, `ILevelupable`, `IUpgradable`, `IProducable`, multiplier, or locatable implementation must deliberately define its audio, locks, groups, callbacks, source-aware changes, and production-round behavior — even when a member intentionally returns an empty collection or `null`. Keep amount and event invariants intact: raise the same events the built-in holders raise, propagate the correct `SourceType` and operation context, and prefer calling public manager methods over invoking holder callbacks directly.

Use the package extension helpers for inputs, outputs, drop tables, locks, and upgrades instead of reimplementing their evaluation. Before release, compare the implementation against Breaking changes (`BreakingChanges.pdf`) for newly required members.

Logging and diagnostics

Managers implement the package `ILogger` contract and route diagnostics through `LogCategory` (`Verbose`, `Warning`, `Critical`). Logs explain rejected input or state — a failed `TryBuyItems` or `TryActivateManager` logs its reason at `Warning` — and must not become part of gameplay control flow. Set a manager's category to `Verbose` while debugging an integration and back to `Critical` for release.

Extend upgrade enums with stable values

C# enums cannot be inherited or reopened from another assembly. You can add members directly to the package's `UpgradeType` and `OverrideUpgradeType` declarations, but those source changes can conflict with new built-in members or be overwritten when upgrading Easy Idle Game. The safer approach is the commonly called **enum overloading** pattern: project code declares a typed constant whose value is outside the built-in range. Runtime multiplier and override queries compare the underlying numeric value, so the constant can be stored in blocks and passed through the normal APIs without modifying package source and is easier to preserve across package upgrades.

```
using EasyIdleGame;

namespace YourStudio.YourGame
{
    public static class GameUpgradeTypes
    {
        public const UpgradeType WarehouseThroughput = (UpgradeType)1100;
        public const OverrideUpgradeType WarehouseCapacity = (OverrideUpgradeType)1100;
    }
}
```

Use a custom multiplier type from an `UpgradeBlock`, `GenericMultiplierBoost`, or manager block, then read it from the system that owns the custom behavior:

```
BigNumber throughput = UpgradesManager.GetMultiplierOfTypeStatic(
    GameUpgradeTypes.WarehouseThroughput);

BigNumber? capacity = UpgradesManager.Instance?.GetOverrideUpgradeOfType(
    GameUpgradeTypes.WarehouseCapacity);
```

Declaring the value does not create behavior by itself. Custom game code must query and apply the multiplier or override. Keep these rules:

- Start project-defined values at `1000` or higher and maintain one numeric registry across all assemblies in the project that share the enum.
- Give every constant an explicit value. Never renumber or reuse it after a serialized asset or save has stored it.
- `UpgradeType` and `OverrideUpgradeType` are separate enum domains, so the same number may be used once in each; values within the same enum must remain unique.
- Keep constants in project-owned code outside the Easy Idle Game installation folder. Directly extending the package enums is possible, but the typed-constant approach avoids maintaining package-source changes and reduces upgrade conflicts.
- The built-in Unity enum popup lists only members compiled into the base enum. For Inspector authoring, provide a project-owned property drawer/editor or assign and migrate the numeric value through code. A custom name/translation layer affects display only and does not change the stored numeric contract.

Keep the registry next to the constants and document the owner and purpose of every allocated value so new project features choose an unused identifier.

Key enums

Enum	Purpose
`ProductionRoundMode`	Live aggregate, snapshot aggregate, or concurrent production rounds
`DropStrategy` / `DropEvaluationMode`	Independent versus weighted selection, and once versus per-instance evaluation
`LockAmountType`	Current, lifetime, bought, or maximum amount comparisons
`SourceType`	Origin carried through contextual effects and source-aware events
`UpgradeType`	Production, speed, cost, XP, duration, drop, cooldown, and specialized multipliers
`UpgradeBlockOperation`	Multiply or add a contextual block value
`UpgradeRuntimeFilter`	`Any`, `Active`, `Offline`, and `Manual` bit flags
`OverrideUpgradeType`	Absolute currency cap, idle-time cap, or business cap
`OverrideUpgradeModifierOperation`	Multiply or add to an existing override source
`UpgradeQueueMode`	Parallel or sequential timed upgrade completion
`ProductionOutputCalculationMode`	Actual, optimistic preview, per-second preview, current per-second preview, or offline calculation
`ProductionBusinessOutputScalingMode` / `ProductionBusinessOutputScalingSource`	Logarithmic/linear scaling driven by owned or producing amount
`MultiplierSource`	Production multiplier sources a business can ignore
`LogCategory`	`Verbose`, `Warning`, or `Critical` manager diagnostics
`AudioCategory`	`SFX`, `BackgroundMusic`, or `UI` routing for `AudioData`

Never reorder or renumber serialized built-in enum values; saved data and assets store the numeric value. Follow Extend upgrade enums with stable values for project-defined `UpgradeType` and `OverrideUpgradeType` values.

Core type index

Area	Primary definitions	Runtime/services
Businesses	`Business`, `BusinessGroup`, `BusinessWrapper`, `BusinessMergeRecipe`, `BusinessUpgradeLevel`	`BusinessHolder`, `BusinessesManager`, `ProductionOutputCalculator`, `ProductionModifierManager`, `ProductionModifierRegistry`
Currencies	`Currency`, `CurrencyGroup`	`CurrencyHolder`, `CurrencyManager`
Managers	`Manager`, `ManagerGroup`	`ManagerHolder`, `ManagersManager`
Upgrades	`Upgrade`, `UpgradeGroup`, `UpgradeBlock`, `OverrideUpgradeBlock`, filters/enums	`UpgradeHolder`, `PendingUpgrade`, `UpgradesManager`, `UpgradeEffectResolver`
Boosts	`Boost`, `BoostGroup`, built-in boost types	`BoostHolder`, `ActiveBoostHolder`, `BoostsManager`
Locations	`Location`, `LocationGroup`	`LocationHolder`, `LocationsManager`
Shop/offers	`ShopItem`, `ShopItemGroup`, `OfferPoolItem`	`ShopItemHolder`, `ShopManager`, `OffersManager`
Progression	`Achievement`, `AchievementGroup`, `CustomStat`, `Reward`, `LevelDataHolder`, `LevelDescription`	`AchievementHolder`, `CustomStatHolder`, `PlayerStats`, `AchievementsManager`, `DailyRewardsManager`
Reset	Prestige configuration fields	`PrestigeData`, `PrestigeResetOptions`, `ResetSummary`, `PrestigeManager`
Persistence	Definition assets in Resources	`SaveFile`, `SaveAndLoad`, `SaveAndLoadSystem<T>`
Offline	Business definitions and holder snapshots	`ProfitCalculator`, `ProfitData`, `ProfitApplicationSummary`
Common models	`Input`, `Output`, `DropTable`, `Lock`, `BuyAmount`, `Metadata`, `BigNumber`	`RewardCalculator`, `EconomyChangedEvents`, `EconomyTextFormatter`

Use this index, the Try-method index, and the current source together when integrating an API that is not covered by a task-oriented guide.

Compatibility APIs

The runtime contains obsolete compatibility members for old assets and custom code, including:

- Flattened `Business.Outputs` and legacy output lists.
- String business groups and old group-based multiplier overloads.
- Legacy business metadata fields.
- `increaseProductionAmountAfterRoundEnd`.
- Legacy merge holder arrays.
- Legacy prestige output lists.

Do not use obsolete members in new code. Follow Breaking changes (`BreakingChanges.pdf`) and Migration guide (`EasyIdleGame > md > MigrationGuide.md`) when updating an older integration.